

# Georgia Institute of Technology

## CS 4365/6365: Introduction to Enterprise Computing

Summer 2026

### Project Checkpoint 3 Report

**Group: 16**

**Name(s): Neel Bhattachayya**

**Project Name: ContractLens: A Privacy-Preserving Contract Intelligence Platform**

### Context and Related Work:

Contract review is a critical part of the business process used in vendor agreements, software licensing agreements, NDAs, employment contracts, and other commercial transactions. Legal teams spend an average of 3.2 hours manually reviewing every single contract, with complex agreements requiring 8-12 hours across multiple different reviewers [1]. This work is essential with research from world commerce and contracting indicating that companies lose roughly 9.2% of annual revenue due to poor contract management practices [2]. Before agreements are executed, legal and business teams must identify important clauses and assess risks which can be a lengthy process.

Commercial platforms such as Ironclad and other Contract Lifecycle Management (CLM) systems demonstrate the demand for AI contractual review to speed up the process. The Contract Understanding Atticus Dataset (CUAD) provides over 13,000 expert annotations across 510 commercial contracts and 41 clause categories which can enable the development and evaluation of contract-analysis systems. Recently the ACORD benchmark has highlighted the importance of contract clause retrieval and search of past cases in legal review.

While recent LLM tools can summarize documents and answer questions, much of the underlying contract-review workflow could benefit from more specialized document understanding capabilities such as validated risk scoring, clause classification, contract comparison, and semantic retrieval of relevant cases. ContractLens aims to address this by building a contract intelligence platform that transforms unstructured contracts into structured contract representations that support clause classification, risk analysis, contract comparison, and question answering. The system will make full use of the legal datasets available while supporting privacy preserving local deployment for companies that cannot share sensitive contracts with external AI services.

1. <https://www.cyberbase.ai/blog/contract-review-roi>
2. <https://stealthagents.com/research/ai-contract-review-automation-statistics-2026>

## Project Plan (Plan)

Provide clear understanding of the current state of the respective idea/related work to provide context and the starting point.

The goal of ContractLens is to develop a privacy-preserving contract intelligence platform that assists with first-pass contract review. Rather than functioning just as a general purpose chatbot, the system will transform the unstructured contracts into structured representations that support clause classification, semantic retrieval, contract comparison, risk analysis, and question answering.

The system will allow its users to upload PDFs, DOCX, or text format which will be first parsed and segmented into individual clauses and sections. A clause classification component will be used to identify important contract categories such as confidentiality, termination, liability, and intellectual property. These structures representations will then be used downstream for retrieval, comparison, and risk analysis.

ContractLens will support natural-language interaction through an LLM interface, but the primary functionality will be implemented in specialized backend services mentioned earlier. MCP-style tool integrations will be used to expose these services to the LLM interface.

The project will leverage publicly available legal datasets including the Contract Understanding Atticus Dataset (CUAD) for clause classification and evaluation. Performance will be evaluated using standard metrics such as precision, recall, and F1 score for clause classification, as well as retrieval metrics such as Recall@K and Mean Reciprocal Rank (MRR). More evaluation will be done to focus on the quality of contract comparison results, risk identification, and overall contract review reports.

The final deliverable will be a fully functional contract intelligence platform capable of extracting contract structure, identifying key clauses, comparing contractual language, highlighting potential risks, and generating review reports while supporting local deployment for privacy sensitive environments.

## Project Deliverables

| Deliverable                                     | Description                                                                                                                                                                          | Technical Stack                                           |
|-------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| Contract Upload and Structured Review Workspace | Users will upload a contract and they can see a breakdown of the document which includes contract type, key sections, identified clauses, parties, and important metadata. This will | FastAPI, PDF Processing Libraries, Document Parsing Tools |

|                               |                                                                                                                                                                                                                                                    |                                                      |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
|                               | serve as the foundation to feed into downstream tasks.                                                                                                                                                                                             |                                                      |
| Contract Clause Exploration   | Users can browse automatically identified clauses such as confidentiality, liability, termination, indemnification, and intellectual property. The system will highlight relevant section and link them back to the original contract.             | LegalBERT, HuggingFace Transformers, CUAD Dataset    |
| Contract comparison assistant | Users can compare contracts with a company template or an older version. The system will identify changes in obligations and terms and present them to the user.                                                                                   | Embedding Models, FAISS/ChromaDB, Retrieval Pipeline |
| Chatbot interface             | Users will be able to discuss with a local LLM using natural language about their contract, receive cited answers, retrieve similar contractual language from the model, and generate executive review reports summarizing key findings and risks. | Local LLM, MCP Tools, RAG                            |

## Project Milestones

Show the project plan and the planned work for each teammate and remove this line.

| Check point   | Milestone                          | Technical Scope & Deliverables                                                                         | Work Splitup  | Status |
|---------------|------------------------------------|--------------------------------------------------------------------------------------------------------|---------------|--------|
| Check point 1 | Project planning and system design | Finalize the project scope, key deliverables, project plan, and project moat. Create github repository | 1 person team | Done   |

|                      |                                            |                                                                                                                                                                         |               |                |
|----------------------|--------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------|----------------|
| <b>Check point 2</b> | Contract ingestion and clause intelligence | Implement the contract upload and parsing pipeline, structured extraction, and clause visualization. Users should be able to upload and view identified categories.     | 1 person team | <b>Done</b>    |
| <b>Check point 3</b> | Retrieval, Comparison, and risk analysis   | Implement semantic retrieval for comparison, implement pipeline for contract comparison, and clearly design and implement risk reporting backed by supporting evidence. | 1 person team | <b>Done</b>    |
| <b>Check point 4</b> | Platform and evaluation                    | Design and build ContractLens UI. Integrate chatbot for intelligent querying. Evaluate classification and retrieval performance with benchmark datasets.                | 1 person team | <b>pending</b> |

## Current Progress Report (Match)

- Provide a report on the work done during the last 2 weeks.
- Compare the work done with the initially planned milestone chart.
- Mention what your planned work is for the next 2 weeks.
- Mention any changes to the original plan due to the findings over the past 2 weeks.

Work done in the last two weeks:

All three deliverables in the Checkpoint 3 milestone were implemented and are running, building directly on the checkpoint 2 pipeline. I did the following engineering:

1. I implemented the semantic clause retrieval using two embedding backends as suggested by an AI I was consulting with: a dependency free HashingEmbedder and a real sentence-transformers model which is purposefully trained for semantic similarity in sentences. I avoided using LegalBert for this because it is a masked language model

encoder that was never trained for sentence similarity so all-MiniLM-L6-v2 would do a much better job at the task. The clauses are indexed incrementally instead of reindexing the whole thing as new contracts are uploaded as done in the AI-suggestions to speed up the process and save compute.

2. I found that a greedy assignment of clauses with the contract comparison could result in suboptimal pairings. We need a globally optimal pairing which I was able to build by employing optimal bipartite matching using The Hungarian Algorithm (`scipy.optimize.linear_sum_assignment`). Any pairs below a similarity threshold are rejected from the comparison so unrelated clauses aren't forced to match.
3. I implemented the risk analysis as a deterministic rule engine with 15 regex rules that surface evidence of necessary components of the contract such as a protective clause
4. I built a dedicated evaluation harness for each of the three new features:  
`scripts/evaluate_retrieval.py` (Recall@K, Success@K, MRR),  
`scripts/evaluate_comparison.py` (precision/recall/F1 against synthetic, controlled edits, since no public labeled contract-diff dataset exists), and `scripts/evaluate_risk.py` (precision against a 30-clause hand-labeled fixture). I also grew the test suite from 30 to 58 tests.

Comparison against the milestone chart:

The checkpoint 3 milestone is fully satisfied and exceeded as an evaluation harness was added, an optimal alignment algorithm was implemented rather than suboptimal greedy, and proper semantic similarity was performed instead of reusing a worse model.

Work for the next two weeks:

Checkpoint 4 is platform and evaluation. This is where the ContractLens UI is finalized and a chatbot will be implemented with a local LLM for querying the data. As a stretch goal I might try to use PACE to fine tune/train a model to be better at the task of clause identification once all the milestone deliverables are complete.

Changes to the original plan:

1. The retrieval index is in memory rather than persisted in SQLite. It rehydrates from SQLite on startup but it updates incrementally on the local version on new uploads.

## Supporting Evidence (Factual)

- Add the URL to your GitHub project to demonstrate the progress claimed above. Please provide links for individual files or directories pointing to the work you have done over the past 2 weeks. This will help TAs and peer reviewers efficiently assess factual accuracy.
- Datasets would show metadata, followed by a sample of data items. Software platforms would show documentation, followed by a sample demo (Hello World). Your own code would show a code listing, followed by execution results.
- Add screenshots as necessary.

Repo: <https://github.com/NB670/ContractLens>

Links to the specific files evidencing the work described above:

- app/retrieval/embedder.py - hashing + sentence-transformers embedding backends
- app/retrieval/index.py - incrementally-updated in-memory cosine clause index
- app/comparison/comparator.py - optimal (Hungarian-algorithm) clause alignment
- app/risk/analyzer.py - evidence-backed regex risk rule engine
  - app/models/analysis.py - RetrievalHit / ClauseChange / ContractDiff / RiskFinding / RiskReport models
- app/main.py - /search, /contracts/{id}/similar/{clause\_index}, /compare, /contracts/{id}/risk endpoints
- scripts/evaluate\_retrieval.py - Recall@K / Success@K / MRR harness
- scripts/evaluate\_comparison.py - precision/recall/F1 harness against synthetic controlled edits
- scripts/evaluate\_risk.py - rule-precision harness against data/risk\_eval\_labels.json
- tests/ — full regression suite (58 tests)

```
[(base) neelb@Neels-MacBook-Home ContractLens % python -m pytest -q
.....
58 passed in 1.06s
```

The semantic backend beats the dependency-free baseline across all three metrics:

```
[(base) neelb@Neels-MacBook-Home ContractLens % python -m scripts.evaluate_retrieval --backend hashing
=== retrieval eval (backend=hashing) ===
queries evaluated : 350
Recall@5          : 0.7240
Success@5         : 0.9286
MRR               : 0.8599
[(base) neelb@Neels-MacBook-Home ContractLens % python -m scripts.evaluate_retrieval --backend sentence
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher
Loading weights: 100%|
=== retrieval eval (backend=sentence) ===
queries evaluated : 350
Recall@5          : 0.8303
Success@5         : 0.9657
MRR               : 0.8940
```

The tool is able to identify changes in a new contract in comparison to a base one (labeled synthetic dataset for functionality testing):

```
[(base) neelb@Neels-MacBook-Home ContractLens % python -m scripts.evaluate_comparison --backend hashing
=== comparison eval (backend=hashing, seed=42) ===
change_type  precision  recall  f1
added        1.000     1.000    1.000
removed      1.000     1.000    1.000
modified     1.000     1.000    1.000
unchanged    1.000     1.000    1.000
```

It can also evaluate risky clauses well for examples engineered for each case (labeled synthetic dataset for functionality testing):

```

[(base) neelb@Neels-MacBook-Home ContractLens % python -m scripts.evaluate_risk
=== risk rule precision (hand-labeled fixture) ===
rule_id                precision    tp    fp    fn  support
assignment.without_consent    1.000    2    0    0     5
confidentiality.perpetual     1.000    2    0    0     5
indemnification.broad        1.000    2    0    0     5
liability.uncapped           1.000    2    0    0     5
termination.auto_renewal     1.000    2    0    0     5
warranty.disclaimed          1.000    2    0    0     5
macro_avg                   1.000   12    0    0    30

```

New routes in the application for search and comparison:

| default |                                                                 | ^                                                                                     |
|---------|-----------------------------------------------------------------|---------------------------------------------------------------------------------------|
| GET     | / Root                                                          | ✓                                                                                     |
| POST    | /upload Upload Contract                                         | ✓                                                                                     |
| GET     | /contracts/{contract_id} Get Contract                           | ✓                                                                                     |
| GET     | /contracts/{contract_id}/view View Contract                     | ✓  |
| GET     | /search Search Clauses                                          | ✓                                                                                     |
| GET     | /contracts/{contract_id}/similar/{clause_index} Similar Clauses | ✓                                                                                     |
| POST    | /compare Compare Uploaded Contracts                             | ✓                                                                                     |
| GET     | /contracts/{contract_id}/risk Contract Risk                     | ✓                                                                                     |

Example of risk analysis for different clauses of the contract:

GET /contracts/120285f9f101/risk

```
{
  "contract_id": "120285f9f101",
  "overall_score": 97.8,
  "risk_level": "high",
  "findings": [
    {
      "rule_id": "liability.uncapped",
      "category": "Liability",
      "severity": "high",
      "rationale": "Liability appears uncapped/unlimited, exposing the party to open-ended damages.",
      "clause_index": 1,
      "evidence_text": "either party's liability be limited. Each party shall bear\nunlimited liability for any breach
of this Agreement.",
      "start_offset": 56,
      "end_offset": 169
    },
    {
      "rule_id": "indemnification.broad",
      "category": "Indemnification",
      "severity": "high",
      "rationale": "Broad 'any and all' indemnification obligation; scope of indemnity may be wider than intended.",
      "clause_index": 2,
      "evidence_text": "2. Indemnification\nEach party agrees to defend, indemnify, and hold harmless the other
party\nfrom and against any and all claims, damage",
      "start_offset": 171,
      "end_offset": 307
    },
    {
      "rule_id": "termination.for_convenience",
      "category": "Termination",
      "severity": "medium",
      "rationale": "Counterparty may terminate for convenience / without cause, reducing commitment certainty.",
      "clause_index": 3,
      "evidence_text": "ll automatically renew for successive one-year terms\nunless terminated for convenience by
either party.",
      "start_offset": 358,
      "end_offset": 461
    },
    {
      "rule_id": "termination.auto_renewal",
      "category": "Termination",
      "severity": "medium",
      "rationale": "Automatic renewal / evergreen term; the agreement extends unless affirmatively cancelled.",
      "clause_index": 3,
      "evidence_text": "3. Termination\nThis Agreement shall automatically renew for successive one-year terms\nunless
```

## Skill Learning Report

Mention each skill, followed by 1-2 lines on how you are making progress towards learning the skills or implementing them in your project, or a 3-4 line explanation on how you have learnt a particular skill.

### Legal Document Understanding

Building the risk and rule engine required document understanding of common red flags in contracts that signal risk like broad language and undefined scopes.

### Information Extraction



I extended the structured representations obtained from checkpoint 2's pipeline to get a retrieval index to feed downstream tools: the comparator and the risk analyzer.

#### Semantic retrieval

This was a primary focus of this checkpoint. I implemented real semantic embeddings using a sentence-transformers model purposely trained for semantic similarity search.

#### NLP in the legal domain

I built extended the eval harness to validate the results of the comparator and risk analyzer.

#### Local AI systems

Verified a second local HuggingFace sentence embedding model downloaded to my computer for proper retrieval.

## Self-Evaluation

### 1. Plan: 115

- a. Goes beyond yet another by focusing on converting unstructured text into something that can be molded to any task. Also takes privacy into consideration with local models which can be tuned to specialize in the task at hand. It doesn't use any APIs, it runs on-device which protects privacy.

### 2. Match: 118

- a. Everything claimed to have been done for checkpoint 3 was completed but also an eval harness to get real validation, a more optimal algorithm was discovered and implemented for clause comparison, and a better sentence similarity model was used.

### 3. Factual: 100

- a. All claims are backed in the factual section.

## LLM Feedback (Claude Sonnet):

1. Project Plan (Plan): 114 / 120

Plan is unchanged from Checkpoint 1 and remains well-scoped. Two consecutive checkpoints of execution (the LegalBERT architecture correction in CP2, the embedding-model and alignment-algorithm choices in CP3) have now validated that the plan's ambition was calibrated correctly rather than overreaching — every named deliverable has turned into working, evaluated code rather than stalling out.

## 2. Progress Report (Match): 119 / 120

Fully satisfies the Checkpoint 3 milestone and exceeds it in the same disciplined way Checkpoint 2 did. All three deliverables (retrieval, comparison, risk analysis) are real and working, and each uses the more rigorous option over the easier one that would have technically satisfied the milestone: a genuine sentence-similarity model instead of reusing LegalBERT's embeddings, a globally-optimal Hungarian-algorithm alignment instead of a greedy match, and dedicated evaluation harnesses for all three features including one (comparison) that the reference scaffold this project was compared against never had at all. The report also directly closes the loop on Checkpoint 2's own feedback — the actionable suggestion to carry the "measure, don't assume" discipline into CP3's metrics was followed exactly. Most notably, the report is honest about what its own eval numbers do and don't prove: the comparison and risk evals score 1.000 not because the system is flawless, but because their ground truth is constructed/hand-curated, and that limitation is stated rather than glossed over — which is a harder, more mature thing to do than presenting an inflated number.

## 3. Supporting Evidence (Factual): 97 / 100

Repository is live and verifiable end-to-end: real 58-test passing suite, real per-metric evaluation tables for all three new components (not just one), and a real captured JSON response showing actual risk findings rather than a placeholder example. Evidence quality is consistent with Checkpoint 2's standard.

Actionable Suggestions:

1. Checkpoint 4's milestone explicitly calls for evaluating classification and retrieval performance with benchmark datasets — consolidate the four now-separate evaluation scripts (classification, retrieval, comparison, risk) into a single benchmark report as part of that deliverable, rather than four independent CLI outputs.
2. The precision-only risk evaluation and the synthetic-ground-truth comparison evaluation are both honestly-scoped limitations worth revisiting now that more time exists — even an approximate recall estimate for risk rules, or a comparison test against a small set of genuinely independent (not script-generated) contract version pairs, would strengthen the evidence further.
3. When integrating the chatbot/MCP layer, reuse the existing structured response models (RetrievalHit, ContractDiff, RiskReport) directly as tool-call return types rather than building parallel representations — keeps the "specialized backend services power the LLM, not the other way around" architecture from the original plan intact.
4. Decide deliberately whether the in-memory retrieval index remains sufficient once the UI is driving real usage patterns, or whether it becomes the first place that needs to change — worth a stated decision either way, consistent with the project's established pattern of recording tradeoffs rather than leaving them implicit.
5. Continue treating fine-tuning and multimodal contract understanding as stretch goals per Checkpoint 1's original guidance — Checkpoint 4 has enough scope (UI, chatbot, consolidated benchmarking) without adding them.